

Ασκήσεις προετοιμασίας (2025-2026)

Αρχές Γλωσσών Προγραμματισμού
Προετοιμασία: Χρήστος Γκόγκος

Κεφάλαιο 1

Άσκηση 1

Γράψτε ένα παράδειγμα που να επιδεικνύει τη διαφορά για τον τελεστή μοναδιαίας αύξησης ++ ανάμεσα στο x++ (postfix) και στο ++x (prefix), στη γλώσσα C.

Απάντηση:

```
#include <stdio.h>

int main() {
    int x = 5;
    int a, b;

    a = x++; // postfix
    printf("a = %d, x = %d\n\n", a, x);

    x = 5;
    b = ++x; // prefix
    printf("b = %d, x = %d\n", b, x);

    return 0;
}
```

a = 5
x = 6
b = 6
x = 6

Άσκηση 2

Ποιες από τις ακόλουθες τιμές θεωρούνται αληθείς και ποιες ψευδείς στη C: 0, 1, -1, 0.0, 3.5, 0.0f, 3.5f, 'a', NULL; Ποιος είναι ο ρόλος του #include <stdbool.h>

Απάντηση:

Αληθείς τιμές: 1, -1, 3.5, 3.5f, 'a'

Ψευδείς τιμές: 0, , 0.0, 0.0f, NULL

Από τη C99 και μετά η <stdbool.h> εισάγει τον τύπο bool και τις σταθερές τιμές : true, false

Άσκηση 3

Με ποιο τρόπο ορίζουμε ψευδώνυμο για μια ακέραια μεταβλητή που ορίζεται ως εξής:

```
int x = 2;
```

στη C και με ποιο στη C++;

Απάντηση:

Στην C μέσω δεικτών:

```
int x = 2;
```

```
int *px = &x; // px είναι ψευδώνυμο μέσω δείκτη
```

```
*px = 5; // αλλάζει και το x
```

Στην C++ μέσω αναφορών (ή μέσω δεικτών όπως στη C):

```
int x = 2;
int &rx = x; // rx είναι πραγματικό ψευδώνυμο του x
rx = 5; // αλλάζει και το x
```

Άσκηση 4

Έστω το ακόλουθο πρόγραμμα:

```
#include <stdio.h>

#define max(A, B) A > B ? A : B // εντολή προεπεξεργασίας

int main(void) {
    double x, y = 3, z = 20;
    x = 2 * max(y, z / 4);
    printf("%.1f\n", x);
}
```

Διορθώστε την εντολή προεπεξεργασίας έτσι ώστε το πρόγραμμα να εμφανίζει το σωστό αποτέλεσμα.

Απάντηση:

```
#define max(A, B) ((A) > (B) ? (A) : (B)) // εντολή προεπεξεργασίας
```

Άσκηση 5

Έστω ένας πηγαίος κώδικας στη C με όνομα my_program.c. Ποια εντολή μεταγλώττισης πρέπει να δοθεί έτσι ώστε να παραχθεί εκτελέσιμο με όνομα my_program.exe;

Απάντηση:

```
> gcc my_program.c -o my_program.exe
```

Άσκηση 6

Δώστε ένα σύντομο παράδειγμα κώδικα σε C στο οποίο να παραβιάζεται το χαρακτηριστικό των γλωσσών προγραμματισμού ορθογωνικότητα και σχολιάστε το.

Απάντηση:

Οι πίνακες μπορούν να έχουν στοιχεία τύπου των τύπων δεδομένων char, int, float, αλλά δεν μπορούν να έχουν στοιχεία τύπου δεδομένων void. Έτσι

```
int a[5]; // ok
```

```
void a[5]; // not ok
```

Άσκηση 7

Δίνεται ο ακόλουθος κώδικας:

```
#include <stdio.h>
void fun(int a, int b[]) {
    a = 1;
    b[0] = 1;
}
int main(void) {
    int a = 0;
    int b[1] = {0};
    fun(a, b);
    printf("%d %d\n", a, b[0]);
}
```

Τι πρόκειται να εμφανίσει κατά την εκτέλεσή του; Εξηγήστε αν παραβιάζεται η ορθογωνικότητα και γιατί;

Απάντηση:

Θα εμφανίσει: 0, 1

Παραβιάζεται η ορθογωνικότητα διότι ενώ φαινομενικά και τα δύο ορίσματα, a και b περνάνε με τον ίδιο τρόπο στη συνάρτηση fun η τιμή του a μένει αμετάβλητη ενώ του b αλλάζει μετά την κλήση της συνάρτησης στη main().

Άσκηση 8

A) Γράψτε μια συνάρτηση σε C που να δέχεται 2 ακέραιες τιμές ως ορίσματα και να επιστρέφει μια δομή με τις τιμές των δύο ορισμάτων αυξημένες η κάθε μια κατά 1. Χρησιμοποιήστε το ακόλουθο πρωτότυπο για τη συνάρτηση.

```
struct my_struct fun(int a, int b);
```

B) Εξηγήστε τι σημαίνει ότι θα παραβιάζεται η ορθογωνικότητα, αν επιχειρηθεί να επιτευχθεί το ίδιο αποτέλεσμα με το ερώτημα A με μια συνάρτηση που θα επιστρέφει 2 ακέραιες τιμές ως έναν πίνακα με το ακόλουθο πρωτότυπο:

```
int[2] fun(int a, int b);
```

Γ) Τροποποιήστε το ερώτημα B έτσι ώστε να λειτουργεί η επιστροφή των 2 τιμών.

Απάντηση:

A)

```
#include <stdio.h>
#include <stdlib.h>

struct my_struct {
    int x;
    int y;
};

struct my_struct fun(int a, int b) {
    struct my_struct tmp;
    tmp.x = a + 1;
    tmp.y = b + 1;
    return tmp;
}

int main(void) {
    int a = 5, b = 10;
    struct my_struct r = fun(a, b);
    printf("%d %d \n", r.x, r.y);
}
```

B)

Στη γλώσσα προγραμματισμού C, οι συναρτήσεις δεν μπορούν να επιστρέφουν πίνακες (παρά μόνο έμμεσα, μέσω δεικτών).

Γ)

```
#include <stdio.h>
#include <stdlib.h>

int *fun(int a, int b) {
    int *tmp = malloc(sizeof(int) * 2);
    tmp[0] = a + 1;
```

```
    tmp[1] = b + 1;
    return tmp;
}

int main(void) {
    int a = 5, b = 10;

    int *r2 = fun(a, b);
    printf("%d %d \n", r2[0], r2[1]);
    free(r2);
}
```

Κεφάλαιο 3

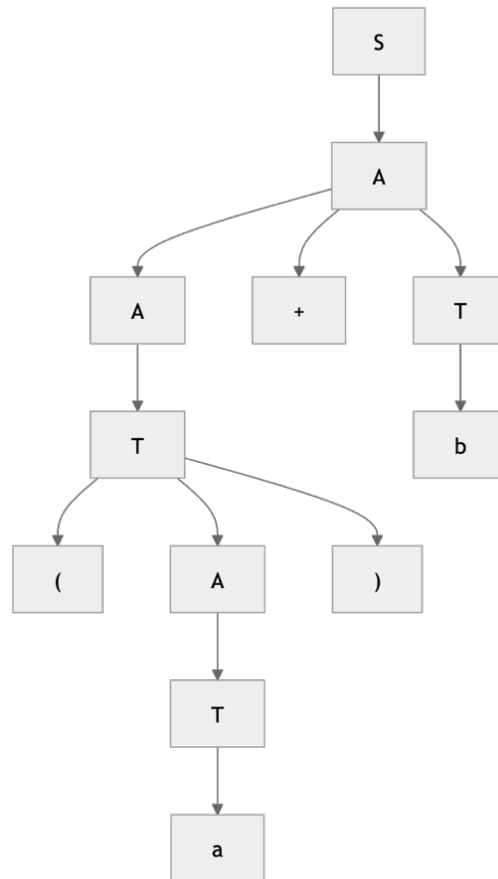
Άσκηση 1

Δίνεται η ακόλουθη γραμματική

$$\begin{aligned} S &\rightarrow A \\ A &\rightarrow T \mid A + T \\ T &\rightarrow a \mid b \mid (A) \end{aligned}$$

Σχεδιάστε ένα δένδρο συντακτικής ανάλυσης για την έκφραση: **(a) + b**

Απάντηση:



Άσκηση 2

Δίνεται η ακόλουθη γραμματική

$$\begin{aligned} S &\rightarrow AaBb \\ A &\rightarrow Ab \mid b \\ B &\rightarrow aB \mid a \end{aligned}$$

Ποιες από τις παρακάτω προτάσεις δεν βρίσκονται στη γλώσσα που παράγει αυτή η γραμματική και γιατί;

1. baab
2. bbbab
3. bbaaaaaS
4. bbaab

Απάντηση:

Η γραμματική ορίζει προτάσεις με 1 ή περισσότερα b, μετά ένα a, μετά ένα ή περισσότερα a και μετά ένα b. Συνεπώς, έγκυρα είναι τα 1 και 4.

Το 2. bbbab, έχει μόνο ένα a

Το 3. bbaaaaaS έχει το μη τερματικό σύμβολο S στο τέλος

Άσκηση 3

Δίνεται η ακόλουθη γραμματική:

$E \rightarrow E + T \mid T$

$T \rightarrow T * F \mid F$

$F \rightarrow (E) \mid a \mid b$

Καταγράψτε μια αριστερότερη παραγωγή που οδηγεί στην έκφραση $a * (a+b)$.

Απάντηση:

$E \Rightarrow T \Rightarrow T * F \Rightarrow F * F \Rightarrow a * F \Rightarrow a * (E) \Rightarrow a * (E + T) \Rightarrow a * (T + T) \Rightarrow a * (F + T) \Rightarrow a * (a + T) \Rightarrow a * (a + b)$

Άσκηση 4

Δίνεται η ακόλουθη γραμματική (σε BNF):

```
<assign> -> <id> = <expr>
<id> -> X | Y | Z
<expr> -> <expr> + <term>
          | <term>
<term> -> <term> * <factor>
          | <factor>
<factor> -> ( <expr> )
           | <id>
```

A. Γράψε μια παραγωγή της πρότασης:

$X = (X + Y * Z) * Z$

B. Σχεδιάστε το δένδρο συντακτικής ανάλυσης για την ίδια πρόταση.

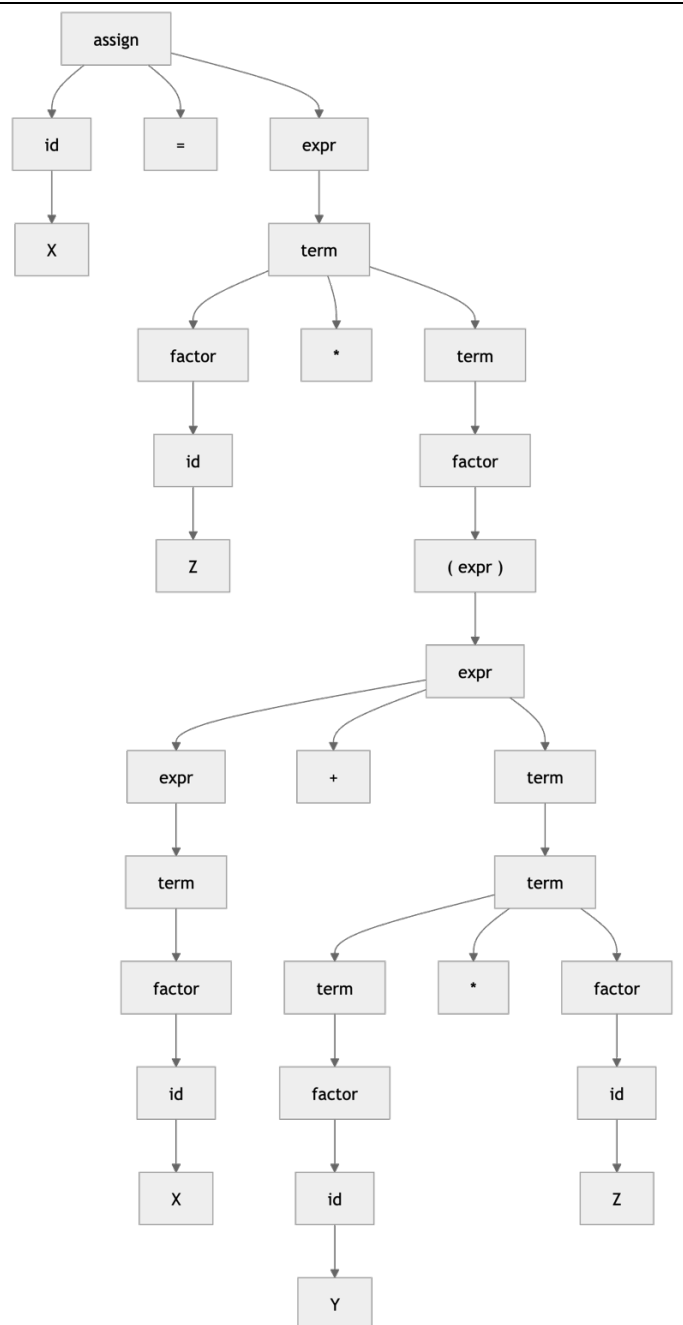
Απάντηση:

Μια παραγωγή (αριστερότερη) της πρότασης είναι η:

```

<assign> ==>
<id> = <expr> ==>
X = <expr> ==>
X = <term> ==>
X = <term> * <factor> ==>
X = <factor> * <factor> ==>
X = (<expr>) * <factor> ==>
X = (<term> + <term>) * <factor> ==>
X = (<term> + <term>) * <factor> ==>
X = (<factor> + <term>) * <factor> ==>
X = (<id> + <term>) * <factor> ==>
X = (X + <term>) * <factor> ==>
X = (X + <term> * <factor>) * <factor> ==>
X = (X + <factor> * <factor>) * <factor> ==>
X = (X + <id> * <factor>) * <factor> ==>
X = (X + Y * <factor>) * <factor> ==>
X = (X + Y * <id>) * <factor> ==>
X = (X + Y * Z) * <factor> ==>
X = (X + Y * Z) * <id> ==>
X = (X + Y * Z) * Z

```



Άσκηση 5

Έστω η ακόλουθη γραμματική (σε BNF):

```

<assign> -> <id> = <expr>
<id>      -> A | B | C
<expr>    -> <id> + <expr>
           | <id> * <expr>
           | ( <expr> )
           | <id>

```

A. Γράψτε μια παραγωγή της πρότασης:

$C = A + B * (C + A)$

B. Σχεδιάστε το δένδρο συντακτικής ανάλυσης για την ίδια πρόταση.

Απάντηση:

<assign>

=> <id> = <expr>

=> C = <expr>

=> C = <id> + <expr>

=> C = A + <expr>

=> C = A + <id> * <expr>

=> C = A + B * <expr>

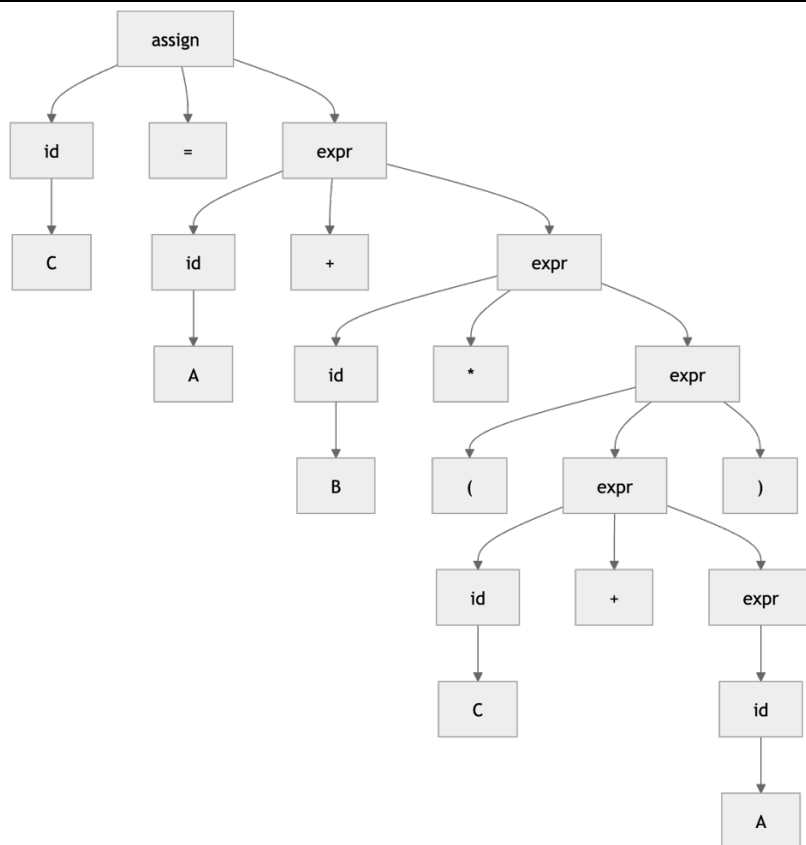
=> C = A + B * (<expr>)

=> C = A + B * (<id> + <expr>)

=> C = A + B * (C + <expr>)

=> C = A + B * (C + <id>)

=> C = A + B * (C + A)



Άσκηση 6

Δίνεται η ακόλουθη γραμματική (σε BNF):

<stmt> -> <matched> | <unmatched> | <non_if_stmt>

<matched> -> if (<logic_expr>) <matched> else <matched>
| <non-if_stmt>

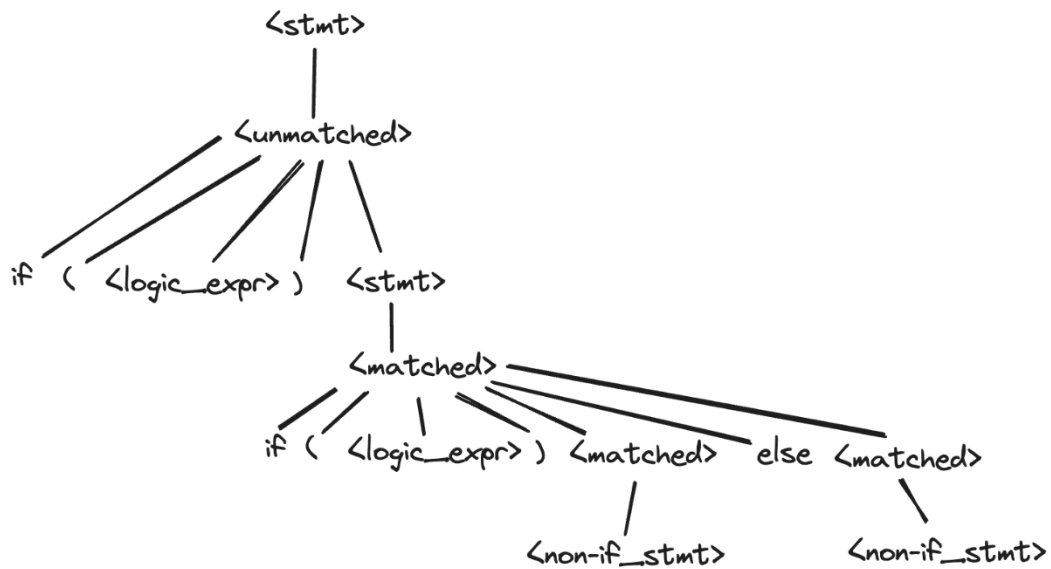
<unmatched> -> if (<logic_expr>) <stmt>

| if (<logic_expr>) <matched> else <unmatched>

1. Τι σημαίνει ότι η γραμματική αυτή επιλύει το πρόβλημα του dangling else;
2. Σχεδιάστε το δένδρο συντακτικής ανάλυσης για την πρόταση:
if (<logic_expr>) if (<logic_expr>) <non_if_stmt> else <non_if_stmt>

Απάντηση:

1. Η γραμματική αυτή επιλύει το πρόβλημα του ξεκρέμαστος else, δηλαδή επιβάλλει σε κάθε else να αφορά το πλησιέστερο προηγούμενο if. Αυτό γίνεται με τη διαφοροποίηση ανάμεσα σε matched και unmatched εντολές if.
2. Το δένδρο συντακτικής ανάλυσης είναι το ακόλουθο:



Άσκηση 7

A) Δείξτε ότι η ακόλουθη γραμματική (σε BNF) είναι ασαφής:

```

<S> -> <A>
<A> -> <A> + <A> | <id>
<id> -> a | b | c
  
```

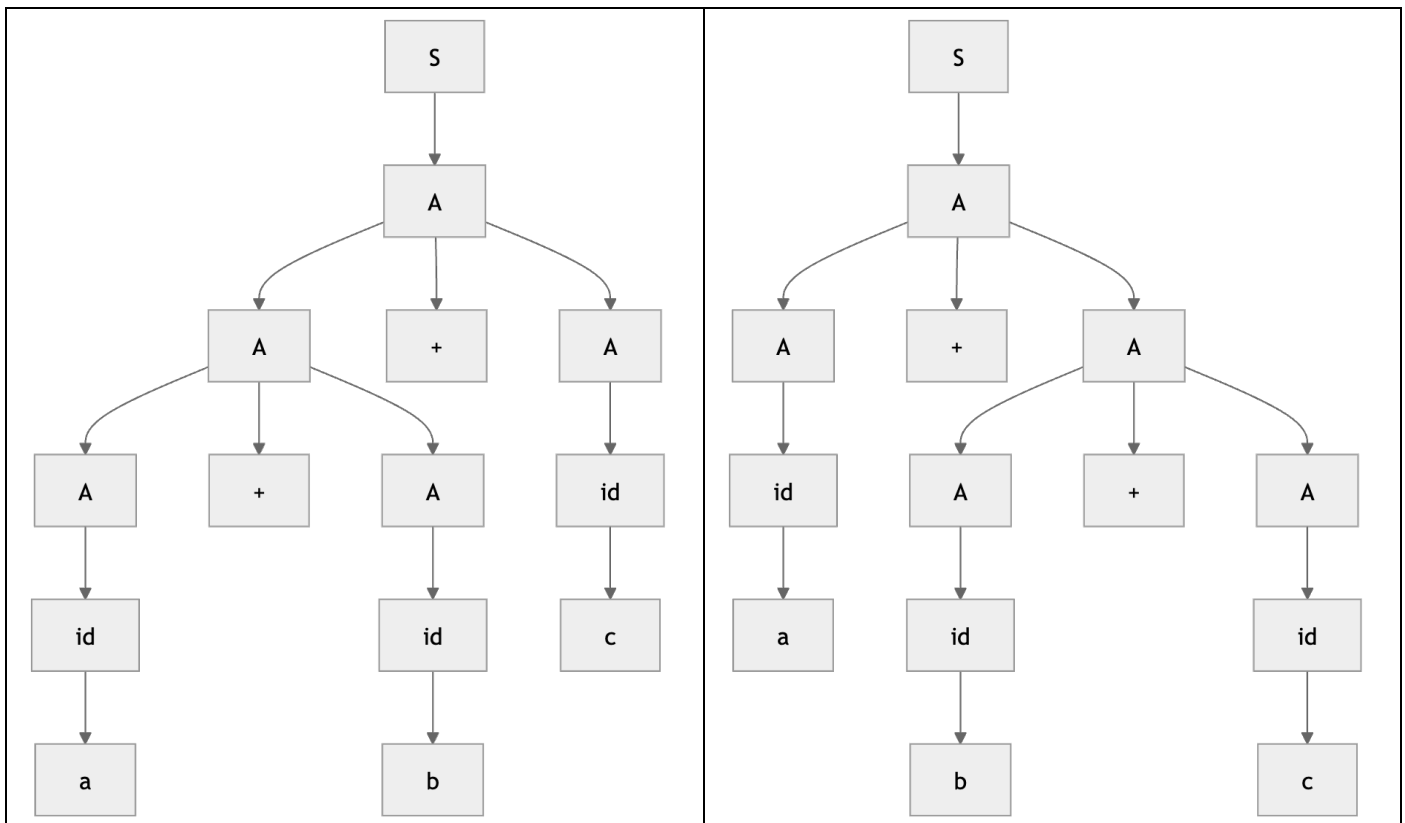
B) Λύνει το πρόβλημα της ασάφειας η ακόλουθη γραμματική;

```

<S> -> <E>
<E> -> <E> + <T> | <T>
<T> -> <id>
<id> -> a | b | c
  
```

Απάντηση:

A) Καθώς προκύπτουν διαφορετικά δένδρα συντακτικής ανάλυσης για την ίδια πρόταση, π.χ. για την πρόταση $a+b+c$, η γραμματική είναι ασαφής.



B) Ναι, η γραμματική είναι σαφής. Στη νέα γραμματική, ο κανόνας:

$E \rightarrow E + T$

επιβάλλει ότι κάθε νέο $+ T$ προστίθεται στα αριστερά ήδη σχηματισμένης έκφρασης. Για την πρόταση $a+b+c$ η μοναδική παραγωγή που μπορεί να υπάρξει είναι η:

$\langle S \rangle \Rightarrow \langle E \rangle \Rightarrow \langle E \rangle + \langle T \rangle \Rightarrow \langle E \rangle + \langle id \rangle \Rightarrow (\langle E \rangle + \langle T \rangle) + \langle id \rangle \Rightarrow (\langle E \rangle + \langle id \rangle) + \langle id \rangle \Rightarrow (\langle id \rangle + \langle id \rangle) + \langle id \rangle \Rightarrow (a + b) + c$

Άσκηση 8

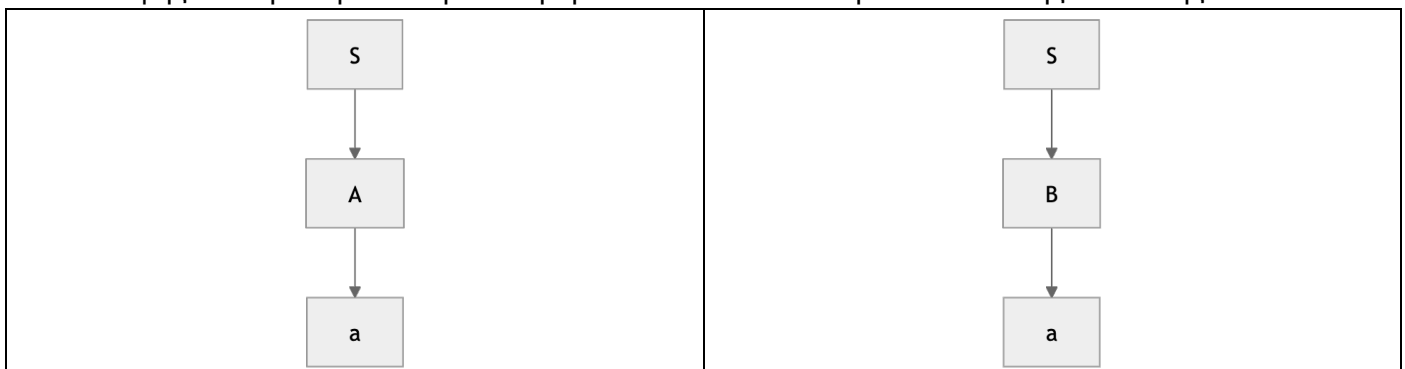
Δίνεται η γραμματική G με τους ακόλουθους κανόνες παραγωγής:

$S \rightarrow A \mid B$
 $A \rightarrow aA \mid a$
 $B \rightarrow aB \mid a$

όπου το αλφάβητο αποτελείται μόνο από το σύμβολο a . Να ελέγξετε αν η παραπάνω γραμματική είναι ασαφής.

Απάντηση:

Είναι ασαφής διότι για την ίδια πρόταση προκύπτουν δύο δένδρα συντακτικής ανάλυσης.



Κεφάλαιο 5

Ερώτημα 1

Τι είναι πρόσδεση (binding); Δώστε ένα παράδειγμα πρόσδεσης στην C και ένα παράδειγμα πρόσδεσης στην Python.

Απάντηση:

Πρόσδεση (binding) είναι η διαδικασία κατά την οποία ένα αναγνωριστικό (identifier) συσχετίζεται με μια οντότητα του προγράμματος, όπως μια τιμή, έναν τύπο δεδομένων, μια διεύθυνση μνήμης.

<pre>#include <stdio.h> int main() { int x = 10; printf("%d\n", x); return 0; }</pre>	<pre>x = 10 print(x) x = "hello" print(x)</pre>
Στατικές προσδέσεις (static binding) στη C Κατά τη μεταγλώττιση το όνομα της μεταβλητής x προσδένεται στον τύπο δεδομένων int. Κατά την εκτέλεση δεσμεύεται μια θέση μνήμης για τη μεταβλητή.	Δυναμικές προσδέσεις (dynamic binding) στη Python Κατά την εκτέλεση η μεταβλητή x προσδένεται στο αντικείμενο 10 και μετά προσδένεται στο αντικείμενο "hello"

Ερώτημα 2

Ποια η διαφορά δήλωσης (declaration) και ορισμού (definition) στη C; Δώστε ένα παράδειγμα με δήλωση/ορισμός μεταβλητής και ένα με δήλωση/ορισμό συνάρτησης.

Απάντηση:

Η δήλωση δηλώνει μια μεταβλητή και καθορίζει τον τύπο της. Δεν δεσμεύει κατ' ανάγκη μνήμη (π.χ. μεταβλητές extern). Ο ορισμός μιας μεταβλητής δημιουργεί πραγματικά τη μεταβλητή δεσμεύοντας μνήμη πιθανά αρχικοποιώντας τη μεταβλητή.

<pre>extern int x; // δήλωση (δεν δεσμεύει μνήμη)</pre>
<pre>int x = 10; // ορισμός (δεσμεύει μνήμη και αρχικοποιεί)</pre>

Η δήλωση μιας συνάρτησης (prototype) καθορίζει το όνομα, τον τύπο επιστροφής και τις παραμέτρους της, χωρίς να περιλαμβάνει το σώμα της. Ο ορισμός της συνάρτησης περιλαμβάνει και το σώμα της, δηλαδή τον κώδικα που εκτελεί.

<pre>int f(int x); // δήλωση (prototype)</pre>
<pre>int f(int x) { // ορισμός return x * 2; }</pre>

Ερώτημα 3

Ποια η διαφορά ανάμεσα στη διάρκεια ζωής (lifetime) και στην εμβέλεια (scope); Δώστε ένα παράδειγμα στη C που να φαίνεται η διαφορά τους.

Απάντηση:

Η εμβέλεια (scope) μιας μεταβλητής είναι το τμήμα του προγράμματος στο οποίο η μεταβλητή είναι ορατή (δηλαδή μπορεί να χρησιμοποιηθεί με το όνομά της). Δηλαδή που φαίνεται μια μεταβλητή. Η διάρκεια ζωής (lifetime) είναι το χρονικό διάστημα κατά την εκτέλεση του προγράμματος στο οποίο η μεταβλητή υπάρχει στη μνήμη. Δηλαδή πόσο υπάρχει στη μνήμη η μεταβλητή.

```
#include <stdio.h>

void f() {
    static int x = 0;
    x++;
    printf("%d\n", x);
}

int main() {
    f();
    f();
    f();
    return 0;
}
```

Η μεταβλητή x έχει εμβέλεια (scope) μόνο μέσα στη συνάρτηση f (δεν είναι ορατή στο main). Η μεταβλητή x έχει διάρκεια ζωής (lifetime) σε όλη τη διάρκεια εκτέλεσης του προγράμματος (λόγω static). Κάθε φορά που καλείται η f(), η x δεν επαναρχικοποιείται, αλλά διατηρεί την τιμή της.

Η έξοδος είναι:

1
2
3

Ερώτημα 4

Έστω ο ακόλουθος κώδικας σε μια υποθετική γλώσσα προγραμματισμού:

```
function outer(){
    function inner1() {
        var x = 5;
        inner2();
    }
    function inner2() {
        var y = x;
        print(y)
    }
    var x = 16;
    inner1();
}
outer()
```

Τι θα εμφανιστεί κατά την εκτέλεση αν η γλώσσα προγραμματισμού διαθέτει α) στατική εμβέλεια, β) δυναμική εμβέλεια;

Απάντηση:

Με στατική εμβέλεια εκτυπώνει 16, με δυναμική εμβέλεια εκτυπώνει 5. Με στατική εμβέλεια η μεταβλητή x που χρησιμοποιείται στη inner2 αναζητείται στο περιβάλλον όπου ορίστηκε η inner2, δηλαδή μέσα στη outer. Εκεί η x έχει τιμή 16, άρα εκτυπώνεται 16. Με δυναμική εμβέλεια η μεταβλητή x αναζητείται στο περιβάλλον κλήσης. Η inner2 καλείται από τη inner1, όπου υπάρχει τοπική x = 5, άρα εκτυπώνεται 5.

Ερώτημα 5

Δίνεται ο ακόλουθος κώδικας σε Python.

```

choice = input("1 για ακέραιους, 2 για πραγματικούς: ")
if choice == "1":
    print("Δώσε 2 ακεραίους: ", end="")
    a, b = input().split()
    print("Αποτέλεσμα: ", int(a) + int(b))
elif choice == "2":
    print("Δώσε 2 πραγματικούς: ", end="")
    a, b = input().split()
    print("Αποτέλεσμα: ", float(a) + float(b))
else:
    print("Λάθος επιλογή.")

```

Γράψτε κώδικα σε C που να έχει ισοδύναμη συμπεριφορά με τον παραπάνω κώδικα. Σε ποια περίπτωση από τους δύο κώδικες έχουμε δυναμική πρόσδεση τύπων και σε ποια στατική πρόσδεση τύπων;

Απάντηση:

Ο ισοδύναμος κώδικας σε C είναι ο ακόλουθος:

```

#include <stdio.h>

int main(void) {
    int choice;
    printf("1 για ακέραιους, 2 για πραγματικούς: ");
    scanf("%d", &choice);

    if (choice == 1) {
        int a, b;
        printf("Δώσε δύο ακεραίους: ");
        scanf("%d %d", &a, &b);
        printf("Αποτέλεσμα: %d\n", a + b);
    } else if (choice == 2) {
        float a, b;
        printf("Δώσε δύο πραγματικούς: ");
        scanf("%f %f", &a, &b);
        printf("Αποτέλεσμα: %.2f\n", a + b);
    } else {
        printf("Λάθος επιλογή\n");
    }

    return 0;
}

```

Στην περίπτωση της Python έχουμε δυναμική πρόσδεση τύπων (κατά το χρόνο εκτέλεσης), ενώ στη C έχουμε στατική πρόσδεση τύπων (κατά την μεταγλώττιση).

Ερώτημα 6

Δίνεται ο ακόλουθος κώδικας σε C. Τι θα συμβεί κατά τη μεταγλώττιση και σύνδεση; Τι σημαίνει ότι η μεταβλητή x είναι extern;

```

#include <stdio.h>

int main() {
    extern int x;
    printf("%d", x);
}

```

Απάντηση:

Η μεταγλώττιση θα προχωρήσει κανονικά, αλλά η σύνδεση θα εμφανίσει μήνυμα σφάλματος καθώς δεν θα μπορεί να εντοπίσει τη μεταβλητή x που έχει οριστεί ως εξωτερική. Θα έπρεπε η μεταβλητή να είχε οριστεί σε ένα άλλο αρχείο πηγαίου κώδικα που να είχε συμπεριληφθεί στην εντολή μεταγλώττισης και σύνδεσης. Εναλλακτικά, η μεταβλητή x θα μπορούσε να είχε οριστεί μετά την main.

Η λέξη-κλειδί `extern` σημαίνει ότι η μεταβλητή δεν ορίζεται στο συγκεκριμένο αρχείο, αλλά έχει οριστεί σε κάποιο άλλο αρχείο (ή σε άλλο σημείο του προγράμματος). Δηλαδή, η δήλωση `extern int x;` απλώς ενημερώνει τον `compiler` για την ύπαρξη της `x`, χωρίς να δεσμεύει μνήμη.

Ερώτημα 7

Τι θα εμφανίσει ο ακόλουθος κώδικας (σε C++) κατά την εκτέλεσή του;

```
#include <iostream>

int x = 100;

void foo() {
    static int x = 0;
    x++;
    std::cout << "foo: " << x << std::endl;
}

void bar() {
    int x = 50;
    x++;
    std::cout << "bar: " << x << std::endl;
}

void baz() {
    x++;
    std::cout << "baz: " << x << std::endl;
}

int main() {
    foo();
    bar();
    baz();
    foo();
    bar();
    baz();
}
```

Απάντηση:

Καθώς οι `static` μεταβλητές διατηρούν τις τιμές τους σε διαδοχικές κλήσεις της συνάρτησης στην οποία έχουν δηλωθεί οι τιμές που θα εμφανιστούν κατά την εκτέλεση θα είναι:

```
foo: 1
bar: 51
baz: 101
foo: 2
bar: 51
baz: 102
```

Ερώτημα 8

Τι θα εμφανίσει ο ακόλουθος κώδικας σε Python; Ποιος είναι ο ρόλος του `_` στο βρόχο `for`;

```
data = [(1, 10), (2, 20), (3, 30)]

for _, value in data:
    print(value)
```

Απάντηση:

Ο κώδικας θα εμφανίσει:

```
10
```

20
30

Η μεταβλητή `_` χρησιμοποιείται ως ανώνυμη μεταβλητή (throwaway variable), δηλαδή δηλώνει ότι η τιμή αποθηκεύεται προσωρινά αλλά δεν μας ενδιαφέρει να τη χρησιμοποιήσουμε. Είναι σύμβαση στην Python για να δείξουμε ότι ένα μέρος των δεδομένων αγνοείται.

Ερώτημα 9

- α) Τι θα εμφανίσει ο ακόλουθος Python κώδικας;
β) Τι θα συμβεί αν προστεθεί η εντολή `global x`, στη συνάρτηση `f()`, πριν την `x=5`;
γ) Τι θα συμβεί αν προστεθεί η εντολή `x=20`, στη συνάρτηση `g()`, μετά την `print(x)`;

```
x = 10

def f():
    x = 5
    print("f:", x)

def g():
    print("g:", x)

f()
g()
print("main:", x)
```

Απάντηση:

- α) Θα εμφανίσει:

```
f: 5
g: 10
main: 10
```

- β) Αν προστεθεί η εντολή `global x` πριν την `x=5`, τότε

```
f: 5
g: 5
main: 5
```

- γ) Αν προστεθεί η εντολή `x=20`, στη συνάρτηση `g()`, μετά την `print(x)`, θα εμφανιστεί σφάλμα:
`UnboundLocalError: local variable 'x' referenced before assignment`

Ερώτημα 10

- α) Τι θα εμφανίσει το πρόγραμμα;
β) Σε ποια μεταβλητή αναφέρεται το `nonlocal x` μέσα στη `inner2`;
γ) Τι σημαίνει LEGB στην Python;

```
x = 0
def outer():
    x = 1
    def inner1():
        x = 2
        def inner2():
            nonlocal x
            x += 1
            print("inner2:", x)
        inner2()
        print("inner1:", x)
    inner1()
```

```
    print("outer:", x)
outer()
print("global:", x)
```

Απάντηση:

α) Θα εμφανίσει:

inner2: 3

inner1: 3

outer: 1

global: 0

β) το nonlocal x μέσα στη inner2() αναφέρεται στην πλησιέστερη enclosing μεταβλητή x. Η πλησιέστερη είναι η x = 2 μέσα στη inner1.

γ) Το LEGB είναι ο κανόνας που χρησιμοποιεί η Python για να αναζητά μεταβλητές (name resolution). Τα αρχικά σημαίνουν:

- L (Local): τοπικό scope της συνάρτησης
- E (Enclosing): εξωτερικά (enclosing) scopes, δηλαδή συναρτήσεις που περιέχουν άλλες συναρτήσεις
- G (Global): καθολικό scope (το αρχείο/module)
- B (Built-in): ενσωματωμένα ονόματα της Python (π.χ. print, len)

Ερώτημα 11

Δίνεται ο ακόλουθος κώδικας

```
#include <iostream>
#define PI 3.14

const int a = 10;
constexpr int b = 20;

int main() {
    int x;
    std::cin >> x;
    const int c = x + 2;
    constexpr int d = 30;

    std::cout << PI << " " << a << " " << b << " " << c << " " << d;
}
```

α) Ποια είδη σταθερών εμφανίζονται και ποιες είναι compile-time και ποιες run-time;

β) Είναι σωστό να γραφεί constexpr int c = x + 2 στη θέση από το const int c = x + 2;

Απάντηση

α) Η σταθερά PI ορίζεται με μακροεντολή (συνεπώς είναι compile time)

Οι a, b, d είναι σταθερές compile-time

Η c είναι σταθερά run-time

β) Δεν είναι σωστό, διότι η x δεν είναι compile-time σταθερά

Κεφάλαιο 6

Ερώτημα 1

Αναφέρατε όλους τους διαφορετικούς τύπους ακέραιους τύπους δεδομένων στη C. Σε ποια χαρακτηριστικά τους διαφέρουν;

Απάντηση:

Στη C οι ακέραιοι τύποι προκύπτουν από συνδυασμούς των βασικών τύπων char, short, int, long, long long και των προσδιοριστών signed, unsigned.

Οι διαφορετικοί ακέραιοι τύποι είναι:

1. char (το αν είναι signed ή unsigned εξαρτάται από την υλοποίηση)
2. signed char
3. unsigned char
4. short ή short int
5. unsigned short ή unsigned short int
6. int
7. unsigned int
8. long ή long int
9. unsigned long ή unsigned long int
10. long long ή long long int
11. unsigned long long ή unsigned long long int

Οι ακέραιοι τύποι διαφέρουν:

- Στο εύρος τιμών που μπορούν να αναπαραστήσουν (το οποίο εξαρτάται από το μέγεθος σε bytes).
- Στο αν είναι προσημασμένοι (signed) ή μη προσημασμένοι (unsigned).

Ερώτημα 2

Γιατί η έκφραση $0.1 + 0.2 \neq 0.3$ επιστρέφει true στις περισσότερες γλώσσες προγραμματισμού; Προτείνετε έναν τρόπο που θα μπορούσε η σύγκριση να επιστρέφει false (π.χ. στη C).

Απάντηση:

Η έκφραση επιστρέφει true επειδή οι δεκαδικοί αριθμοί δεν αναπαρίστανται ακριβώς σε δυαδική μορφή κινητής υποδιαστολής, με αποτέλεσμα μικρά σφάλματα στρογγυλοποίησης που οδηγούν τη σύγκριση $0.1 + 0.2 \neq 0.3$ στην τιμή true.

Ένας τρόπος ώστε η σύγκριση να επιστρέφει false είναι να μην συγκρίνουμε απευθείας για ισότητα, αλλά να ελέγχουμε αν η διαφορά των δύο τιμών είναι μικρότερη από ένα πολύ μικρό όριο:

$\text{fabs}((0.1 + 0.2) - 0.3) < 1e-9$

Ερώτημα 3

Τι είναι το IEEE 754;

Απάντηση:

Το **IEEE 754** είναι ένα διεθνές πρότυπο που καθορίζει την αναπαράσταση και τους υπολογισμούς αριθμών κινητής υποδιαστολής στους υπολογιστές, όπου κάθε αριθμός αποθηκεύεται σε δυαδική μορφή αποτελούμενη από τρία βασικά μέρη: το **signum (bit προσήμου)** που δηλώνει αν ο αριθμός είναι θετικός ή αρνητικός, τον **εκθέτη (exponent)** που καθορίζει τη δύναμη του 2 με την οποία κλιμακώνεται ο αριθμός, και τη **mantissa (ή significand)** που περιέχει τα σημαντικά ψηφία της τιμής· το πρότυπο

ορίζει επίσης μορφές όπως float και double, κανόνες στρογγυλοποίησης και ειδικές τιμές όπως άπειρο και NaN, εξασφαλίζοντας συνεπή και προβλέψιμη αριθμητική συμπεριφορά.

Ερώτημα 4

Στην Python ποια είναι η διαφορά ανάμεσα στα ακόλουθα:

<code>print(0.1+0.2)</code>	<code>from decimal import Decimal print(Decimal("0.1") + Decimal("0.2"))</code>
-----------------------------	---

Απάντηση:

Η διαφορά είναι ότι η έκφραση `print(0.1+0.2)` χρησιμοποιεί αριθμούς κινητής υποδιαστολής τύπου float που ακολουθούν το πρότυπο IEEE 754 και δεν αναπαριστούν ακριβώς τους δεκαδικούς αριθμούς σε δυαδική μορφή, με αποτέλεσμα να εμφανίζεται τιμή όπως 0.30000000000000004, ενώ η έκφραση `print(Decimal("0.1") + Decimal("0.2"))` χρησιμοποιεί τον τύπο Decimal που αναπαριστά τους αριθμούς με δεκαδική ακρίβεια (base-10) και εκτελεί πράξεις χωρίς σφάλματα στρογγυλοποίησης, επιστρέφοντας ακριβώς 0.3.

Ερώτημα 5

Πως γίνεται η σύγκριση αλφαριθμητικών στη C, στη C++ και στην Python;

Απάντηση:

Η σύγκριση δύο αλφαριθμητικών ως προς το ποιο προηγείται γίνεται λεξικογραφικά, δηλαδή χαρακτήρα - χαρακτήρα με βάση τους κωδικούς των χαρακτήρων (συνήθως ASCII ή Unicode):

- Στη C χρησιμοποιείται η συνάρτηση `strcmp(s1, s2)`, η οποία επιστρέφει αρνητική τιμή αν το s1 προηγείται του s2, μηδέν αν είναι ίσα και θετική αν έπεται.
- Στη C++, όταν χρησιμοποιούνται αντικείμενα `std::string`, μπορούν να χρησιμοποιηθούν απευθείας οι τελεστές σύγκρισης όπως `<`, `>`, `==`, που επίσης κάνουν λεξικογραφική σύγκριση. Στην Python οι συμβολοσειρές συγκρίνονται απευθείας με τελεστές όπως `<`, `>`, `==`, με λεξικογραφικό τρόπο βάσει των κωδικών Unicode των χαρακτήρων.

Ερώτημα 6

Να γράψετε πρόγραμμα σε C, C++ και Python στο οποίο να ορίζεται ένας τύπος απαρίθμησης (Enumeration / Enum) για τις εργάσιμες ημέρες της εβδομάδας με τιμές MONDAY, TUESDAY, WEDNESDAY, THURSDAY, FRIDAY. Το πρόγραμμα να δηλώνει μία μεταβλητή αυτού του τύπου, να της αναθέτει κατάλληλη τιμή. Να δοθεί λύση και στις τρεις γλώσσες, με χρήση του αντίστοιχου μηχανισμού απαρίθμησης που υποστηρίζει καθεμία. Για την C++ να χρησιμοποιηθεί `enum class`.

Απάντηση:

<pre>#include <stdio.h> enum Day { MONDAY, TUESDAY, WEDNESDAY, THURSDAY, FRIDAY }; int main() { enum Day d; d = WEDNESDAY; printf("%d\n", d); return 0; }</pre>	<pre>#include <iostream> using namespace std; enum class Day { MONDAY, TUESDAY, WEDNESDAY, THURSDAY, FRIDAY }; int main() { Day d = Day::WEDNESDAY; cout << static_cast<int>(d) << endl; return 0; }</pre>	<pre>from enum import Enum class Day(Enum): MONDAY = 1 TUESDAY = 2 WEDNESDAY = 3 THURSDAY = 4 FRIDAY = 5 d = Day.WEDNESDAY print(d)</pre>
C	C++	Python

Ερώτημα 7

Δημιουργήστε έναν οδοντωτό πίνακα 10 σειρών στη C που να έχει στην 1^η σειρά 1 στοιχείο στη 2^η σειρά 2 στοιχεία κ.ο.κ.

Απάντηση:

```
#include <stdio.h>
#include <stdlib.h>

int main() {
    int i, j;

    int **arr = malloc(10 * sizeof(int*));

    for (i = 0; i < 10; i++) {
        arr[i] = malloc((i + 1) * sizeof(int));
    }

    for (i = 0; i < 10; i++) {
        for (j = 0; j <= i; j++) {
            arr[i][j] = j;
        }
    }

    for (i = 0; i < 10; i++) {
        for (j = 0; j <= i; j++) {
            printf("%d ", arr[i][j]);
        }
        printf("\n");
    }

    for (i = 0; i < 10; i++) {
        free(arr[i]);
    }
    free(arr);

    return 0;
}
```

Ερώτημα 8

Δίνεται η λίστα:

L = [10, 20, 30, 40, 50]

Ποια θα είναι τα αποτελέσματα από τους ακόλουθους τεμαχισμούς:

1. L[1:4]
2. L[:3]
3. L[2:]
4. L[-3:]
5. L[:-1]
6. L[-4:-1]
7. L[:,2]
8. L[1::2]
9. L[:, -1]
10. L[0:10]

Απάντηση:

1. $L[1:4] \rightarrow [20, 30, 40]$
2. $L[:3] \rightarrow [10, 20, 30]$
3. $L[2:] \rightarrow [30, 40, 50]$
4. $L[-3:] \rightarrow [30, 40, 50]$
5. $L[:-1] \rightarrow [10, 20, 30, 40]$
6. $L[-4:-1] \rightarrow [20, 30, 40]$
7. $L[::2] \rightarrow [10, 30, 50]$
8. $L[1::2] \rightarrow [20, 40]$
9. $L[::-1] \rightarrow [50, 40, 30, 20, 10]$
10. $L[0:10] \rightarrow [10, 20, 30, 40, 50]$ (δεν προκαλείται σφάλμα)

Ερώτημα 9

Να εξετάσετε τον τρόπο ελέγχου εύρους δεικτών σε πίνακες (ή λίστες) στις γλώσσες C, C++, Java και Python με βάση τον πίνακα [1, 2, 3, 4]. Να εξετάσετε τι συμβαίνει κατά την πρόσβαση σε μία έγκυρη θέση (π.χ. δείκτη 2) και σε μία μη έγκυρη θέση (π.χ. δείκτη 5).

Απάντηση:

Για τον πίνακα {1, 2, 3, 4} στη C, C++ και Java και για τη λίστα [1,2,3,4] στην Python η πρόσβαση στη θέση με δείκτη 2 είναι έγκυρη και επιστρέφει την τιμή 3, αφού οι δείκτες αρχίζουν από το 0. Η διαφορά εμφανίζεται στην πρόσβαση στη μη έγκυρη θέση με δείκτη 5.

- Στη C δεν γίνεται αυτόματος έλεγχος ορίων στους πίνακες, επομένως η πρόσβαση είναι απροσδιόριστη συμπεριφορά (Undefined Behavior): μπορεί να εμφανιστεί τυχαία τιμή, να προκληθεί σφάλμα ή να φανεί ότι το πρόγραμμα «λειτουργεί» λανθασμένα.
- Στην C++, όταν χρησιμοποιούνται κλασικοί πίνακες με τον τελεστή [], ισχύει ουσιαστικά το ίδιο με τη C, δηλαδή δεν υπάρχει αυτόματος έλεγχος ορίων και η συμπεριφορά είναι επίσης απροσδιόριστη. Μόνο σε δομές όπως `std::vector` με χρήση της μεθόδου `at()` υπάρχει έλεγχος.
- Στη Java οι πίνακες ελέγχονται αυτόματα ως προς τα όρια και η πρόσβαση στη θέση 5 προκαλεί εξαίρεση `ArrayIndexOutOfBoundsException` κατά την εκτέλεση.
- Στην Python οι λίστες ελέγχονται αυτόματα και η πρόσβαση στη θέση 5 προκαλεί εξαίρεση `IndexError`.

Ερώτημα 10

Πως γίνεται στη C η αρχικοποίηση ενός πίνακα που περιέχει τα στοιχεία "Arta", "Ioannina", "Preveza";

Απάντηση:

```
char *cities[] = {"Arta", "Ioannina", "Preveza"};
ή
char cities[][10] = {"Arta", "Ioannina", "Preveza"};
```

Ερώτημα 11

Έστω ένας δισδιάστατος πίνακας διαστάσεων 3 x 4 με στοιχεία ακέραιων αριθμών (με αρίθμηση γραμμών και στηλών που ξεκινά από 0). Να γράψετε τη γραμμική διάταξη των στοιχείων στη μνήμη για row-major και column-major διάταξη αποθήκευσης και να προσδιορίσετε τη θέση (offset) του στοιχείου στη θέση (1,2) σε κάθε περίπτωση. Ποια από τις δύο διατάξεις χρησιμοποιεί η κάθε μια από τις ακόλουθες γλώσσες προγραμματισμού: C, Fortran, Java, MATLAB.

Απάντηση:

Έστω πίνακας 3×4 με δείκτες από 0: $A = [[a_{00}, a_{01}, a_{02}, a_{03}], [a_{10}, a_{11}, a_{12}, a_{13}], [a_{20}, a_{21}, a_{22}, a_{23}]]$. Στη row-major διάταξη τα στοιχεία αποθηκεύονται γραμμή-γραμμή: $a_{00}, a_{01}, a_{02}, a_{03}, a_{10}, a_{11}, a_{12}, a_{13}, a_{20}, a_{21}, a_{22}, a_{23}$, και το offset του στοιχείου στη θέση (1,2) δίνεται από τον τύπο $i*4 + j = 1*4 + 2 = 6$. Στη column-major διάταξη τα στοιχεία αποθηκεύονται στήλη-στήλη: $a_{00}, a_{10}, a_{20}, a_{01}, a_{11}, a_{21}, a_{02}, a_{12}, a_{22}, a_{03}, a_{13}, a_{23}$, και το offset του στοιχείου (1,2) είναι $j*3 + i = 2*3 + 1 = 7$. Η C και η Java χρησιμοποιούν row-major διάταξη, ενώ η Fortran και η MATLAB χρησιμοποιούν column-major.

Ερώτημα 12

Να γράψετε πρόγραμμα σε Python και σε C++ στο οποίο να χρησιμοποιείται συσχετιστικός πίνακας για την αποθήκευση βαθμών φοιτητών, όπου ως κλειδί χρησιμοποιείται το όνομα και ως τιμή ο βαθμός. Ο συσχετιστικός πίνακας να περιλαμβάνει τα ζεύγη "Μαρία" → 7.5 και "Νίκος" → 6.0. Το πρόγραμμα να εμφανίζει τον βαθμό ενός συγκεκριμένου φοιτητή με βάση το όνομα που θα δίνει ο χρήστης. Να δοθεί λύση στην Python με χρήση dict και στη C++ με χρήση unordered_map.

Απάντηση:

<pre> grades = { "Μαρία": 7.5, "Νίκος": 6.0 } name = input() if name in grades: print(grades[name]) else: print("Not found") </pre>	<pre> #include <iostream> #include <unordered_map> using namespace std; int main() { unordered_map<string, double> grades; grades["Μαρία"] = 7.5; grades["Νίκος"] = 6.0; string name; cin >> name; if (grades.find(name) != grades.end()) { cout << grades[name] << endl; } else { cout << "Not found" << endl; } return 0; } </pre>
Python	C++

Ερώτημα 13

Να γράψετε περιφραστικές λίστες (list comprehensions) στην Python και στην Haskell που να υπολογίζουν τα ακόλουθα:

1. Λίστα με τα τετράγωνα των αριθμών από το 1 έως το 10.
2. Λίστα που να περιέχει μόνο τους άρτιους αριθμούς από το 1 έως το 20.
3. Λίστα με τους αριθμούς από το 1 έως το 10 που είναι πολλαπλάσια του 3.
4. Λίστα με όλα τα ζεύγη (x, y) όπου x και y παίρνουν τιμές από το 1 έως το 3.
5. Λίστα που να περιέχει τα τετράγωνα μόνο των περιττών αριθμών από το 1 έως το 15.

Απάντηση:

<pre> # 1 [x*x for x in range(1, 11)] # 2 [x for x in range(1, 21) if x % 2 == 0] # 3 </pre>	<pre> -- 1 [x^2 x <- [1..10]] -- 2 [x x <- [1..20], x `mod` 2 == 0] -- 3 </pre>
--	---

<pre>[x for x in range(1, 11) if x % 3 == 0] # 4 [(x, y) for x in range(1, 4) for y in range(1, 4)] # 5 [x*x for x in range(1, 16) if x % 2 == 1]</pre>	<pre>[x x <- [1..10], x `mod` 3 == 0] -- 4 [(x, y) x <- [1..3], y <- [1..3]] -- 5 [x^2 x <- [1..15], x `mod` 2 == 1]</pre>
Python	Haskell

Ερώτημα 14

Να θεωρήσετε έναν τύπο δεδομένων `union` στη γλώσσα C που μπορεί να αποθηκεύει είτε έναν ακέραιο (`int`), είτε έναν πραγματικό (`float`), είτε έναν χαρακτήρα (`char`). Να δηλώσετε μια μεταβλητή αυτού του `union`, να της αναθέσετε διαδοχικά τιμές διαφορετικών τύπων. Να εξηγήσετε γιατί η τιμή που είχε αποθηκευτεί προηγουμένως χάνεται και να σχολιάσετε τη βασική διαφορά του `union` από μια `struct`.

Απάντηση:

<pre>union Data { int i; float f; char c; };</pre>
<pre>union Data d;</pre>

Αν εκτελέσουμε διαδοχικά αναθέσεις όπως `d.i = 10;`, στη συνέχεια `d.f = 3.14;` και μετά `d.c = 'A';`, παρατηρούμε ότι κάθε νέα ανάθεση «καταστρέφει» την προηγούμενη τιμή. Αυτό συμβαίνει επειδή όλα τα μέλη του `union` μοιράζονται την ίδια περιοχή μνήμης και κάθε φορά που γράφουμε σε ένα μέλος, αντικαθιστούμε τα δεδομένα των άλλων. Αντίθετα, σε μια `struct` κάθε πεδίο έχει τη δική του ανεξάρτητη θέση στη μνήμη, οπότε όλες οι τιμές μπορούν να συνυπάρχουν ταυτόχρονα. Συνεπώς, το `union` χρησιμοποιείται όταν θέλουμε να αποθηκεύσουμε μία μόνο τιμή από ένα σύνολο πιθανών τύπων, εξοικονομώντας μνήμη.

Ερώτημα 15

Δώστε 3 παραδείγματα σχεδιαστικών αποφάσεων της Java έτσι ώστε να είναι ασφαλέστερη γλώσσα από τη C++

Απάντηση:

Η Java:

1. Δεν επιτρέπει άμεση χρήση δεικτών και χειροκίνητη διαχείριση μνήμης από τον προγραμματιστή (υπάρχει αυτόματη διαχείριση μνήμης μέσω `garbage collection`).
2. Παρέχει αυτόματο έλεγχο ορίων στους πίνακες (`array bounds checking`), αποτρέποντας προσβάσεις εκτός ορίων.
3. Δεν υποστηρίζει `union`, αποφεύγοντας επικίνδυνες επικαλύψεις μνήμης.

Ερώτημα 16

Να μελετήσετε το ακόλουθο πρόγραμμα σε C:

<pre>#include <stdio.h> int main() { int a = 10, b = 20; int *p; p = &a; printf("%d\n", *p);</pre>
--

```

*p = 15;
printf("%d %d\n", a, b);

p = &b;
*p = 25;
printf("%d %d\n", a, b);

return 0;
}

```

Να απαντήσετε στα ακόλουθα:

1. Ποιες τιμές εκτυπώνονται σε κάθε εντολή printf;
2. Τι περιέχει ο δείκτης p μετά από κάθε ανάθεση (p = &a, p = &b);
3. Ποια είναι η διαφορά μεταξύ των εκφράσεων p και *p;
4. Πώς επηρεάζει η αποαναφορά (*p) τις μεταβλητές a και b;
5. Τι θα συμβεί αν αφαιρεθεί η εντολή p = &b και παραμείνει η εντολή *p = 25;

Απάντηση:

1. Εκτυπώνει:

```

10
15 20
15 25

```

2. Περιεχόμενο του p:

Μετά το p = &a → διεύθυνση της a

Μετά το p = &b → διεύθυνση της b

3. Το p είναι διεύθυνση μνήμης ενώ το *p είναι η τιμή στη διεύθυνση αυτή

4. ο *p = ... αλλάζει την τιμή της μεταβλητής στην οποία δείχνει ο δείκτης

5. Αν αφαιρεθεί το p = &b τότε η εντολή *p = 25 θα αλλάξει την τιμή του a και θα εκτυπωθεί : a = 25, b = 20

Ερώτημα 17

Δώστε ένα παράδειγμα με dangling pointer στη C, δηλαδή έναν δείκτη που δείχνει σε μια heap-dynamic μεταβλητή που έχει αποδεσμευτεί.

Απάντηση:

```

#include <stdio.h>
#include <stdlib.h>

int main() {
    int *p = malloc(sizeof(int));
    *p = 42;
    free(p);
    printf("%d\n", *p);
    return 0;
}

```

Ερώτημα 18

Γράψτε ένα παράδειγμα κώδικα που να προκαλεί διαρροή μνήμης (memory leakage) στη C.

Απάντηση:

```
#include <stdio.h>
#include <stdlib.h>

int main() {
    int *p = malloc(10 * sizeof(int));

    for (int i = 0; i < 10; i++) {
        p[i] = i;
    }

    p = malloc(20 * sizeof(int)); // memory leak

    return 0;
}
```

Ερώτημα 19

Τι θα εκτυπώσει ο ακόλουθος κώδικας C:

```
#include <stdio.h>

int main() {
    int arr[] = {10, 20, 30, 40, 50};
    int *p = arr;

    printf("%d\n", *p);
    printf("%d\n", *(p + 2));
    printf("%d\n", *(arr + 4));

    p = p + 3;
    printf("%d\n", *p);

    return 0;
}
```

Απάντηση:

Θα εκτυπώσει:

10
30
50
40

Ερώτημα 20

Τι θα εκτυπώσει ο ακόλουθος κώδικας C++ που χρησιμοποιεί αναφορές (references);

```
#include <iostream>
using namespace std;

void update(int &x, int y) {
    x = x + 10;
    y = y + 10;
}
```



```
int main() {  
    int a = 5;  
    int b = 7;  
  
    int &r = a;  
  
    r = 20;  
    update(a, b);  
  
    cout << "a = " << a << endl;  
    cout << "b = " << b << endl;  
    cout << "r = " << r << endl;  
  
    return 0;  
}
```

Απάντηση:

Θα εκτυπώσει:

a = 30

b = 7

r = 30

Κεφάλαιο 7

Ερώτημα 1

Γράψτε ένα παράδειγμα αριθμητικής έκφρασης στην Python που να χρησιμοποιεί τελεστές και τελεστέους, παρενθέσεις και κλήσεις συναρτήσεων. Καταγράψτε τους τελεστές και τους τελεστέους που χρησιμοποιήσατε.

Απάντηση:

Αριθμητική έκφραση: $3 + x ** 2 / (y + \max(x,y))$

Τελεστές: +, **, +

Τελεστέοι: x, y

Ερώτημα 2

Τι θα εκτυπώσει ο ακόλουθος κώδικας:

```
#include <stdio.h>

int main() {
    int x = 5;
    int y = -x++;
    int z = --x;

    printf("x=%d, y=%d, z=%d\n", x, y, z);

    return 0;
}
```

Απάντηση:

Θα εκτυπώσει:

x=5, y=-5, z=5

Ο τελεστής ++, έχει μεγαλύτερη προτεραιότητα από τον μοναδιαίο τελεστή -, άρα -x++ είναι ισοδύναμο με -(x++).

Ερώτημα 3

Να μετατρέψετε τον παρακάτω κώδικα C σε ισοδύναμο κώδικα Python, χρησιμοποιώντας την αντίστοιχη μορφή του τριαδικού τελεστή (ternary operator) στην Python.

```
#include <stdio.h>

int main() {
    int x = 12;
    int result = (x % 2 == 0) ? 1 : 0;
    printf("%d\n", result);
    return 0;
}
```

Απάντηση:

```
x = 12
result = 1 if x % 2 == 0 else 0
print(result)
```

Ερώτημα 4

Μετατρέψτε τον ακόλουθο κώδικα C σε ισοδύναμο κώδικα έτσι ώστε να μην χρησιμοποιεί τριαδικούς τελεστές αλλά μόνο εντολές if-else.

```
#include <stdio.h>

int main() {
    int x = 10, y = 5, z = 8;
    int result = (x > y) ? ((y > z) ? y : z) : ((x > z) ? x : z);
    printf("%d\n", result);
    return 0;
}
```

Απάντηση:

```
#include <stdio.h>

int main() {
    int x = 10, y = 5, z = 8;
    int result;

    if (x > y) {
        if (y > z) {
            result = y;
        } else {
            result = z;
        }
    } else {
        if (x > z) {
            result = x;
        } else {
            result = z;
        }
    }

    printf("%d\n", result);

    return 0;
}
```

Ερώτημα 5

Ποιο είναι το αποτέλεσμα των ακόλουθων αριθμητικών εκφράσεων στην Python;

1. $3 * (10 // 3) + 10 \% 3$
2. $100 / 4 / 2$
3. $2^{**}2^{**}3$
4. $\text{True} * 2 + \text{False} * 3$
5. $10 / 2$

Απάντηση:

1. 10
2. 12.5
3. 256
4. 3
5. 5.0

Ερώτημα 6

Συμπληρώστε τον ακόλουθο κώδικα στη C έτσι ώστε να παραβιάζεται η αναφορική διαφάνεια (referential transparency).

```
#include <stdio.h>

...
int fun(int x) {
    ...
}

int main() {
    int a = 5, b = 10, c = 2;
    int result1 = (fun(a) + b) / (fun(a) - c);
    int temp = fun(a);
    int result2 = (temp + b) / (temp - c);
    printf("result1 = %d\n", result1);
    printf("result2 = %d\n", result2);
    return 0;
}
```

Απάντηση:

```
#include <stdio.h>

int counter = 0;

int fun(int x) {
    return counter++;
}

int main() {
    int a = 5, b = 10, c = 2;
    int result1 = (fun(a) + b) / (fun(a) - c);
    int temp = fun(a);
    int result2 = (temp + b) / (temp - c);
    printf("result1 = %d\n", result1);
    printf("result2 = %d\n", result2);
    return 0;
}
```

Θα εκτυπώσει:

result1 = -10

result2 = 0

Ερώτημα 7

Δίνεται ο ακόλουθος κώδικας:

```
#include <stdio.h>

int main() {
    int a = 10;
    double b = 3.7;
    char c = 'A';
    double x = a + b;    // (1)
    int y = b;           // (2)
    int z = c + a;       // (3)
    char k = a + 200;    // (4)
```

```
printf("x=%.2f, y=%d, z=%d, k=%c\n", x, y, z, k);
return 0;
}
```

Σε ποιες περιπτώσεις έχουμε widening και σε ποιες narrowing μετατροπές και ποια είναι τα προβλήματα που ενδεχόμενα θα παρουσιαστούν;

Απάντηση:

Στο (1) έχουμε widening για το a (int->double).

Στο (2) έχουμε narrowing για το b (double ->int), αποκόπτεται το δεκαδικό μέρος, η μεταβλητή y λαμβάνει την τιμή 3.

Στο (3) έχουμε widening για το c (char->int), γίνεται η πράξη 65 + 10 -> 75.

Στο (4) έχουμε narrowing για το a + 200 από int σε char, συμβαίνει overflow.

Ερώτημα 8

Ποια θα είναι η συμπεριφορά των ακόλουθων προγραμμάτων;

<pre>#include <stdio.h> int main() { int a = 5; int b = 0; int c = a / b; int d = b / b; printf("c=%d, d=%d\n", c, d); return 0; }</pre>	<pre>#include <stdio.h> int main() { float a = 5.0f; float b = 0.0f; float c = a / b; float d = b / b; printf("c=%f, d=%f\n", c, d); return 0; }</pre>
A	B

Απάντηση:

Στο A) το αποτέλεσμα θα είναι απροσδιόριστο διότι συμβαίνει undefined behavior.

Στο B) το αποτέλεσμα θα είναι:

c=inf, d=nan

διότι το IEEE754 ορίζει τη συμπεριφορά της διαίρεσης με το μηδέν για τους αριθμούς κινητής υποδιαστολής

+/0 -> +inf (infinity)

-/0 -> -inf (infinity)

0/0 -> nan (not a number)

Ερώτημα 9

Ποια θα είναι τα αποτελέσματα εκτέλεσης του ακόλουθου κώδικα σε C;

<pre>#include <stdio.h> int main() { printf("%s\n", 4 <= 3 <= 5 ? "true" : "false"); printf("%s\n", 4 <= 3 && 3 <= 5 ? "true" : "false"); return 0; }</pre>
--

Απάντηση:

Θα εμφανίσει:

true διότι 4 <= 3 αποτιμάται σε 0 και στη συνέχεια 0 <= 5 αποτιμάται σε 1 που είναι αληθές, άρα εκτυπώνει true.

false διότι 4 <= 3 είναι ψευδές και 3 <= 5 είναι αληθές, άρα εκτυπώνεται false.

Ερώτημα 10

Ποια θα είναι η συμπεριφορά του ακόλουθου κώδικα C; Αν υπάρχει πρόβλημα πως μπορεί να διορθωθεί;

```
int x = 42;  
x = x + ++x;  
printf("%d\n", x);
```

Απάντηση:

Ο κώδικας παρουσιάζει undefined behavior (απροσδιόριστη συμπεριφορά) διότι στην έκφραση: $x + ++x$ η γλώσσα C δεν καθορίζει ποια από τα 2 θα γίνει πρώτα: η αύξηση του x με το $++$ ή η λήψη τιμής για το x ως αριστερός τελεστέος της έκφρασης: $x + ++x$.

Το πρόβλημα undefined behavior του κώδικα διορθώνεται για παράδειγμα με τον ακόλουθο κώδικα με έναν από τους δύο ακόλουθους τρόπους ανάλογα με το τι θέλει να πετύχει ο προγραμματιστής:

```
int x = 42;  
++x;  
x = x + x;  
printf("%d\n", x);
```

```
int x = 42;  
x = x + x;  
++x;  
printf("%d\n", x);
```